СИСТЕМНЫЙ ДИЗАЙН-ИНТЕРВЬЮ · АРХИТЕКТУРА ИНФОРМАЦИОННЫХ СИСТЕМ

Проектирование архитектуры маркетплейса с собственным фулфилментом и механикой Flash Sales

Пошаговое проектирование по методологии System Design Interview: функциональные и нефункциональные требования, back-of-the-envelope оценка нагрузки, микросервисная архитектура (Database per Service), глубокий разбор узла защиты от оверселлинга при горячих распродажах, ER-модель данных и анализ CAP-компромиссов.

Этап 0. Выбор предметной области

> **Рассуждение кандидата вслух на старте интервью:**
> *«Давайте перед тем, как рисовать квадратики и стрелочки, чётко зафиксируем бизнес-модель нашего интернет-магазина. Обычная витрина с одной базой данных не представляет архитектурной интриги. Поэтому я предлагаю спроектировать федеральный маркетплейс по модели FBO (Fulfillment by Operator — как Ozon, Wildberries или Яндекс Маркет), где площадка хранит товары независимых селлеров на собственных региональных складах-фулфилментах, сама собирает заказы, доставляет их курьерами и регулярно проводит горячие распродажи (Flash Sales) с экстремальными пиками нагрузки».*

Описание системы и бизнес-контекст. Выбранная система — крупный интернет-маркетплейс с собственным складским фулфилментом, курьерской доставкой «последней мили» и поддержкой лимитированных распродаж (Flash Sales). В отличие от простой доски объявлений, такая система объединяет в единую цепочку четыре разных контура: высоконагруженную витрину для покупателей, кабинет управления ассортиментом и поставками для продавцов, систему управления адресным складом (WMS) для комплектовщиков и гео-сервис маршрутизации для курьеров.

Ключевые участники (роли пользователей) в системе

- **1. Покупатель (Клиент):** ищет товары в каталоге, изучает карточки, формирует корзину, оформляет и оплачивает заказ, отслеживает статус сборки и движение курьера на карте.
- **2. Продавец (Селлер / Поставщик):** размещает карточки товаров (SKU), управляет ценами и скидками, отгружает партии товара на склады маркетплейса, запускает участие в Flash-акциях и получает аналитику и выплаты.
- **3. Сотрудник склада фулфилмента (Комплектовщик):** работает с терминалом сбора данных (ТСД), принимает поставки селлеров, раскладывает товары по адресным ячейкам, собирает оплаченные заказы по оптимальному маршруту и передаёт их в зону отгрузки.
- **4. Курьер службы доставки:** забирает упакованные заказы с регионального склада или даркстора, доставляет их покупателю (до двери или в ПВЗ), передаёт GPS-координаты в реальном времени и подтверждает вручение.

Почему эта система интересна с точки зрения System Design

- **Сильная асимметрия чтения и записи (Read-to-Write ~ 100:1):** на 1 оформленный заказ приходится около 100 просмотров каталога и поисковых запросов. Витрина должна быть максимально быстрой и доступной, даже если подсистема оформления заказов перегружена.
- **Конфликт за физический ресурс (Inventory Contention & Overselling):** в момент старта Flash Sale десятки тысяч пользователей одновременно пытаются купить 500–1 000 акционных смартфонов. Продажа одного физического экземпляра двум покупателям (оверселлинг) недопустима.
- **Длинные распределённые бизнес-транзакции:** оформление заказа затрагивает сразу несколько независимых сервисов (Заказы, Складской резерв, Эквайринг, Сборка, Доставка), что требует координации через паттерн Saga без глобальных блокировок.

Этап 1. Функциональные требования

> **Рассуждение кандидата вслух:**
> *«Уточним функциональный периметр системы. Я сгруппирую требования по четырём ролям пользователей и сформулирую их через конкретные бизнес-действия, не привязываясь к кнопкам интерфейса. Всего выделим 15 ключевых сценариев, покрывающих полный цикл жизни товара — от поставки селлером на склад до вручения покупателю».*

Роль 1. Покупатель (Клиент)

- **ФТ-1. Поиск и фильтрация каталога:** Покупатель ищет товары по текстовому запросу и категории с фильтрацией по цене, бренду, рейтингу и сроку доставки в его город.
- **ФТ-2. Просмотр карточки товара:** Покупатель просматривает детальную информацию о товаре (характеристики, фото, актуальную цену с учётом акций, доступный остаток на ближайшем складе и отзывы).
- **ФТ-3. Управление корзиной и расчёт доставки:** Покупатель добавляет товары в корзину, применяет промокоды и выбирает адрес и временной интервал (слот) доставки.
- **ФТ-4. Оформление заказа и временный резерв стока:** Покупатель оформляет заказ, при этом выбранные единицы товара временно резервируются за ним на складе фулфилмента на время оплаты (10 минут).
- **ФТ-5. Онлайн-оплата заказа:** Покупатель оплачивает заказ банковской картой или через СБП и получает электронный кассовый чек.
- **ФТ-6. Отслеживание заказа и оценка:** Покупатель отслеживает смену статусов заказа (В сборке → У курьера → Доставлен), видит текущую гео-позицию курьера на карте и оставляет отзыв о товаре после получения.

Роль 2. Продавец (Селлер / Мерчант)

- **ФТ-7. Управление карточками товаров (SKU):** Продавец создаёт и редактирует карточки товаров, загружает фотографии, задаёт габариты, вес и штрихкоды.
- **ФТ-8. Управление ценами и участие во Flash Sales:** Продавец изменяет базовую цену товара и регистрирует товар в «горячей распродаже» с указанием акционной цены и выделенного лимита штук.
- **ФТ-9. Оформление заявки на поставку (Inbound):** Продавец создаёт заявку на отгрузку партии товаров на выбранный региональный склад фулфилмента и бронирует тайм-слот разгрузки.
- **ФТ-10. Просмотр аналитики продаж и выплат:** Продавец просматривает дашборд с динамикой заказов, остатками по складам, удержанной комиссией маркетплейса и суммой к выплате.

Роль 3. Сотрудник склада фулфилмента (Комплектовщик)

- **ФТ-11. Приёмка и размещение товара по ячейкам:** Сотрудник склада сканирует штрихкоды прибывшей поставки продавца и размещает единицы товара в конкретные адресные ячейки хранения (bin locations).
- **ФТ-12. Сборка оплаченного заказа (Picking):** Сотрудник склада получает в ТСД-терминале задание на сборку заказа и собирает зарезервированные товары из ячеек по рассчитанному маршруту обхода склада.
- **ФТ-13. Упаковка и передача в доставку (Packing):** Сотрудник склада упаковывает заказ в коробку, клеит штрихкод отправления и переводит заказ в статус «Готов к выдаче курьеру».

Роль 4. Курьер службы доставки

- **ФТ-14. Приёмка маршрутного листа и погрузка:** Курьер получает сформированный маршрутный лист на смену, сканирует коробки в зоне отгрузки склада и начинает движение по маршруту.
- **ФТ-15. Доставка и подтверждение вручения:** Курьер передаёт фоновые координаты своего движения и вручает заказ покупателю после проверки проверочного кода из клиентского приложения.

Границы проектирования (Out of Scope). Чтобы сфокусироваться на ядре архитектуры на интервью, явно оставим за рамками (Out of Scope): внутреннюю бухгалтерскую отчётность (ERP/1C), рекламный движок аукциона баннеров, чат со службой поддержки и таможенное оформление кросс-бордер заказов.

Этап 2. Нефункциональные требования (NFR)

> **Рассуждение кандидата вслух:**
> *«Нефункциональные требования никогда не берутся с потолка — каждое из них защищает бизнес от*

> *конкретной потери денег или репутации. Давайте пройдёмся по нашим ключевым сценариям и зададим себе вопрос: что случится с бизнесом, если этот узел ответит медленно, окажется недоступен или вернёт неконсистентные данные? Из этого сразу родятся точные метрики SLA, перцентили задержки (p95/p99) и гарантии консистентности».*

Сценарий	«Что если...» (технический сбой / деградация)	Последствие для бизнеса	Измеримое требование (NFR)
1. Поиск товаров и просмотр каталога покупателем	...поисковая выдача и карточки товаров открываются дольше 3–4 секунд?	Рост показателя отказов (Bounce Rate): по исследованиям e-commerce каждые лишние 500 мс задержки снижают конверсию в покупку на 5–7%; покупатель уходит к конкуренту.	Latency поиска и чтения карточки SKU: < 200 мс (p95) и < 400 мс (p99). Доступность витрины каталога (SLA): ≥ 99.99% (не более 52 мин простоя в год).
2. Резервирование товара при нажатии «Оформить заказ»	...произойдёт состояние гонки (race condition) и система продаст 1 последний смартфон сразу 50 покупателям?	Массовые принудительные отмены уже оплаченных заказов (Overselling), штрафы за возврат эквайринга, жалобы в соцсетях и отток лояльных покупателей.	Строгая консистентность остатков (Strong Consistency, 0% overselling). Время атомарного резерва стока при чекауте: < 150 мс (p95) и < 300 мс (p99).
3. Старт горячей распродажи (Flash Sale, например 11.11)	...в первую секунду акции трафик вырастет в 20–50 раз и положит базу данных всего маркетплейса?	Срыв дорогостоящей маркетинговой кампании и полная остановка продаж даже обычных (неакционных) товаров на всей площадке; прямые убытки в десятки млн руб./час.	Система должна выдерживать пик до 50 000 RPS на чтение и до 10 000 RPS на попытки резерва без деградации базового чекаута. Изоляция контура Flash Sale (Bulkhead).
4. Онлайн-оплата заказа и списание средств	...при повторном нажатии кнопки из-за медленного интернета деньги спишутся дважды или оплата потеряется?	Двойное списание средств у клиента, чарджбэки со стороны банков, нарушение 54-ФЗ (онлайн-кассы) и блокировка мерчант-аккаунта у платёжного провайдера.	100% идемпотентность платежных операций (Idempotency Key). Надёжность сохранения транзакций: RPO = 0 (нулевая потеря данных), RTO < 5 мин. SLA платежей ≥ 99.95%.
5. Брошенные неоплаченные заказы на распродаже	...покупатели или боты зарезервируют весь акционный сток в заказах, но не оплатят их?	Товар числится «нет в наличии» для реальных покупателей с деньгами, а после окончания акции остаётся непроданным на складе продавца («заморозка» стока).	Автоматическое снятие неоплаченного резерва по истечении TTL = 10 минут с возвратом единицы товара в доступный пул продажи с задержкой ≤ 5 секунд.
6. Обновление цены или описания товара продавцом	...новая цена или описание появятся в поисковом индексе не мгновенно, а через пару секунд?	Никакого критического ущерба для бизнеса нет, если задержка составляет несколько секунд (покупатель на витрине переживёт секундное отставание индекса).	Допустима итоговая согласованность (Eventual Consistency) для поискового индекса и каталога: время распространения изменений (Replication / Indexing Lag) < 3 сек (p95).
7. Получение задания на сборку на складе (WMS) и трекинг курьера	...координаты курьера на карте клиента обновятся с задержкой или один GPS-пакет потеряется в тоннеле?	Потеря одного GPS-пакета не страшна (придёт следующий через 5 сек), но задержка передачи оплаченного заказа на склад остановит работу	Доставка события заказа в WMS склада: гарантированная (At-least-once) за < 2 сек (p95). Обновление гео-позиции курьера: каждые 5 сек, отклик чтения трека < 100 мс (p95).

		комплектовщиков.	
8. Отказ одного узла БД или целой зоны доступности (AZ)	...выйдет из строя мастер-сервер БД заказов или пропадёт связь с одним из дата-центров?	Остановка приёма заказов по всей стране или потеря информации о заказах, которые уже оплачены клиентами и собираются на складах.	Отказоустойчивость уровня Multi-AZ: синхронная репликация для БД платежей и заказов (RPO = 0), автоматический фейловер мастера за время RTO < 30 секунд.

Этап 3. Оценка нагрузки (Back-of-the-Envelope Estimation)

> **Рассуждение кандидата вслух:**
> *«Теперь переведём наши требования в конкретные цифры трафика, дисков, оперативной памяти и серверов. На интервью главное — прозрачная логика: явно называем исходное бизнес-допущение, округляем числа для удобного счёта через степени десятки (например, 1 сутки ≈ 86 400 сек ≈ 10^5 сек) и проверяем, где именно система упрётся в потолок».*

3.1. Количество пользователей и запросов в секунду (QPS / RPS)

- **Исходные допущения:** аудитория маркетплейса составляет MAU = 30 млн уникальных пользователей в месяц, DAU = 10 млн активных пользователей в сутки. Активный каталог насчитывает 50 млн SKU от 150 тыс. продавцов. В среднем 1 активный пользователь совершает 40 действий чтения в день (поиск, переходы по категориям, открытие карточек товаров, проверка отзывов) и делает в среднем 0,2 заказа в сутки (то есть в сутки оформляется 2 млн заказов). Для удобства расчётов примем 1 сутки = 86 400 сек ≈ 100 000 сек (10^5 сек). Вечерний суточный пик превышает среднюю нагрузку в 3 раза (Peak Factor = ×3), а минутный всплеск при старте Flash Sale даёт десятикратный скачок поверх вечернего пика.
- **Расчёт потока чтения (Read QPS — каталог, поиск, карточки, трекинг):** Суточное число запросов на чтение = 10 000 000 DAU × 40 запросов = 400 000 000 (4×10^8) запросов/сутки. Средний Read QPS = 4×10^8 / 10^5 сек = 4 000 RPS (при точном делении на 86 400 сек ≈ 4 630 RPS). Вечерний пик Read QPS (×3) = 4 630 × 3 ≈ 14 000 RPS. Экстремальный пик в первую минуту старта крупной распродажи (Flash Sale) = до 50 000 Read RPS.
- **Расчёт потока записи (Write QPS — корзина, заказы, резерв стока, гео-трекинг):** 1) Оформление заказов (Checkout + Резерв стока + Оплата): 2 000 000 заказов/сутки / 86 400 сек ≈ 23 заказа/сек в среднем; в вечерний пик (×3) ≈ 70 заказов/сек; однако в момент старта Flash Sale (когда 50 000 человек одновременно жмут «Купить») пиковый поток попыток резерва стока достигает 10 000 Write RPS. 2) Изменение корзины (добавление/удаление товаров — примерно 10 кликов на 1 заказ): 20 000 000 операций/сутки ≈ 230 RPS в среднем и ~700 RPS в вечерний пик. 3) Фоновый гео-трекинг курьеров: на линии одновременно работают 15 000 активных курьеров, каждый приложение отправляет GPS-пинг раз в 5 секунд → 15 000 / 5 = 3 000 Write RPS (постоянный поток записи координат).

Результат по RPS/QPS: Средний трафик API составляет ~4 630 Read RPS и ~3 250 Write RPS (из них 3 000 RPS — лёгкие телеметрические пинги курьеров и ~250 RPS — бизнес-транзакции). В вечерний пик система обслуживает ~14 000 Read RPS и ~3 800 Write RPS, а в момент старта Flash Sale должна выдерживать до 50 000 Read RPS и до 10 000 RPS на резервирование остатков.

3.2. Объём хранимых данных (Storage) — в сутки, в месяц, за год

- **Исходные допущения по размеру записей:** 1) Заказ (заголовок Order + в среднем 3 строки OrderItem + запись Payment + индексы БД) занимает в среднем 2 КБ. 2) Карточка товара (метаданные в PostgreSQL + документ в Elasticsearch) занимает 10 КБ; прирост новых SKU составляет +50 000 товаров в сутки. 3) Медиафайлы (фотографии новых товаров в Object Storage S3): в среднем 5 фото по 200 КБ (с учётом сжатия WebP) = 1 МБ на 1 новый SKU. 4) События гео-трекинга и смены статусов (для аналитики и разбора инцидентов): 3 000 пингов/сек × 100 байт (в сжатом колоночном формате ClickHouse).
- **Расчёт прироста данных:** • Транзакционные данные заказов и платежей (PostgreSQL): 2 000 000 заказов/сутки × 2 КБ = 4 ГБ в сутки → 120 ГБ в месяц → 1,46 ТБ в год (с учётом коэффициента B-Tree индексов и служебных полей ×1,5 ≈ 2,2 ТБ/год). • Каталог и поисковый индекс (PostgreSQL + Elasticsearch): стартовая база 50 000 000 SKU × 10 КБ = 500 ГБ; ежедневный прирост: 50 000 SKU × 10 КБ = 0,5 ГБ/сутки → 15 ГБ/месяц → 180 ГБ/год. •

Логи трекинга курьеров и событий аналитики (ClickHouse): 3 000 записей/сек × 86 400 сек × 100 байт ≈ 26 ГБ в сутки → 780 ГБ в месяц → 9,5 ТБ в год. • Медиафайлы товаров (S3 Object Storage): стартовый объём 50 млн SKU × 1 МБ = 50 ТБ; прирост: 50 000 новых SKU/сутки × 1 МБ = 50 ГБ в сутки → 1,5 ТБ в месяц → 18 ТБ в год.

Результат по объёму данных: Прирост реляционных и поисковых БД составляет ~4,5 ГБ/сутки (~135 ГБ/месяц, ~2,4 ТБ/год с индексами). Прирост аналитических логов и треков — 26 ГБ/сутки (~9,5 ТБ/год). Прирост медиахранилища S3 — 50 ГБ/сутки (~1,5 ТБ/месяц, ~18 ТБ/год). Суммарный прирост всех хранилищ — около 80,5 ГБ в сутки, 2,4 ТБ в месяц и ~30 ТБ в год.

3.3. Пропускная способность сети (Network Bandwidth)

- **Исходные допущения:** средний размер JSON-ответа API при поиске или открытии карточки товара (с учётом gzip/brotli сжатия) = 15 КБ. Кроме JSON-ответов, клиентское приложение подгружает в среднем 6 превью-изображений по 40 КБ (240 КБ статики с CDN на один запрос просмотра). Средний размер входящего запроса (POST/GET payload, GPS-координаты, корзина) = 2 КБ.
- **Расчёт исходящего и входящего трафика:** • Исходящий трафик бэкенд-API (JSON): в среднем 4 630 RPS × 15 КБ ≈ 70 МБ/сек (560 Мбит/с); в вечерний пик (14 000 RPS) = 14 000 × 15 КБ ≈ 210 МБ/сек (1,68 Гбит/с); в пике Flash Sale (50 000 RPS) ≈ 750 МБ/сек (6 Гбит/с). • Исходящий трафик статики и фото (раздаётся с краевых узлов CDN мимо нашего ядра): в среднем 4 630 RPS × 240 КБ ≈ 1,1 ГБ/сек (8,8 Гбит/с); в вечерний пик ≈ 3,36 ГБ/сек (~27 Гбит/с). • Входящий трафик (Ingress API): в среднем (4 630 + 3 250) RPS × 2 КБ ≈ 16 МБ/сек (128 Мбит/с); в пике Flash Sale (60 000 суммарных RPS × 2 КБ) ≈ 120 МБ/сек (~1 Гбит/с).

Результат по сети (Bandwidth): Ядро бэкенда (API Gateway) требует полосы пропускания ~700 Мбит/с в среднем, ~2 Гбит/с в вечерний пик и до 7 Гбит/с во время Flash Sale (легко покрывается парой балансировщиков с сетевыми картами 10/25 GbE). Трафик картинок (~27 Гбит/с в пике) на 95%+ терминируется на внешней сети CDN.

3.4. Требования к оперативной памяти и кэшу (RAM / Redis)

- **Исходные допущения (правило Парето 20/80):** 20% самых популярных товаров генерируют 80% всего трафика просмотров и покупок. Кроме того, в оперативной памяти (Redis) мы храним: 1) карточки «горячего» каталога и их остатки по регионам, 2) активные корзины и сессии пользователей (DAU = 10 млн), 3) атомарные счётчики резервов для товаров и 4) текущие гео-координаты 15 000 курьеров.
- **Расчёт объёма кэша:** • Кэш горячего каталога (20% от 50 млн SKU = 10 млн SKU): 10 000 000 × 8 КБ (сжатый JSON карточки + цены) = 80 ГБ. • Активные корзины и пользовательские сессии (10 млн DAU + ~5 млн гостевых корзин за неделю): 15 000 000 × 2 КБ = 30 ГБ. • Быстрые счётчики складских остатков и временных резервов (50 млн SKU × в среднем 3 склада × 64 байта на ключ в Redis): 150 000 000 × 64 Б ≈ 9,6 ГБ (~10 ГБ). • Гео-индекс активных курьеров и статусы доставок: 15 000 курьеров × 1 КБ ≈ 15 МБ (пренебрежимо мало, < 1 ГБ). • Суммарный полезный объём данных в памяти = 80 + 30 + 10 = 120 ГБ. С учётом накладных расходов структур данных Redis, фрагментации аллокатора jemalloc (×1,3) и репликации Master-Replica (×2 для отказоустойчивости): 120 ГБ × 1,3 × 2 ≈ 312 ГБ RAM.

Результат по памяти (RAM): Для хранения всех горячих данных, корзин и атомарных счётчиков остатков с репликацией потребуется кластер Redis суммарным объёмом ~320–384 ГБ RAM (например, 6 шардов Master + 6 узлов Replica по 32 ГБ RAM каждый).

3.5. Примерное количество серверов (Capacity Planning)

- **Исходные допущения по производительности одного узла:** современный типовой виртуальный узел для stateless-микросервисов (8 vCPU, 16 ГБ RAM, Go/Java) уверенно обрабатывает ~2 500 RPS средней бизнес-логики (с учётом походов в кэш и сериализации JSON) при загрузке CPU не выше 65%. Один узел базы данных PostgreSQL (16 vCPU, 64 ГБ RAM, NVMe SSD) держит ~3 000 простых транзакций записи/чтения в секунду. Один шард Redis (32 ГБ RAM) обслуживает до 80 000 простых операций GET/DECR в секунду.
- **Расчёт количества серверов по слоям:** 1) Stateless-микросервисы (Каталог, Корзина, Заказы, Склад, Платежи, Трекинг): для штатного вечернего пика (~18 000 суммарных RPS) требуется 18 000 / 2 500 ≈ 8 узлов; с учётом резервирования N+2 по зонам доступности — 12 базовых нод в постоянном пуле. В часы проведения Flash Sale (до 60 000 RPS) Kubernetes Horizontal Pod Autoscaler (HPA) автоматически масштабирует пул stateless-сервисов до 60 000 / 2 500 = 24–30 подов/серверов. 2) Кластер кэширования и резерва остатков (Redis Cluster): по объёму памяти (312 ГБ) и по пиковому RPS (60 000+ операций/сек) достаточно 12 инстансов по 32 ГБ RAM (6 Master + 6 Replica). 3) Кластер реляционных БД (PostgreSQL для Заказов, Склада, Платежей и Мастер-каталога): 4 логических шарда для Заказов и Склада + выделенный кластер Платежей + кластер Мастер-каталога = 6 пар (1 Master + 2 Read Replica в разных AZ) = 18 серверов БД. 4) Поисковый кластер (Elasticsearch):

для хранения 500 ГБ индекса с фактором репликации 2 и обслуживания 14 000 поисковых RPS с фильтрами (агрегации CPU-емкие: ~1 500 RPS на узел) требуется 10–12 поисковых узлов (16 vCPU, 64 ГБ RAM). 5) Брокер сообщений (Apache Kafka) + Аналитика (ClickHouse) + API Gateway: 5 брокеров Kafka + 3 узла ClickHouse + 4 узла L7 API Gateway = 12 инфраструктурных серверов.

Результат по серверам: В штатном режиме вся платформа работает на ~64 серверах (12 stateless app-нод, 12 нод Redis, 18 нод PostgreSQL, 10 нод Elasticsearch, 12 инфраструктурных узлов Kafka/Gateway/ClickHouse), а во время Flash Sale пул stateless-приложений эластично расширяется до ~82 серверов.

Метрика нагрузки / Ресурс	Среднее значение	Вечерний пик (×3)	Пик при Flash Sale	Итоговый сайзинг инфраструктуры
Запросы чтения (Read RPS)	4 630 RPS	14 000 RPS	50 000 RPS	Кэш Redis + CDN + 10 узлов Elasticsearch
Запросы записи (Write RPS)	3 250 RPS (3k GPS + 250 зак.)	3 800 RPS	10 000 RPS (резервы)	Шарды Redis + буфер Kafka + PostgreSQL
Прирост хранилища (Storage)	80,5 ГБ / сутки	2,4 ТБ / месяц	30 ТБ / год (с медиа S3)	NVMe SSD для БД + объектное хранилище S3
Полоса сети (Bandwidth)	560 Мбит/с (API)	1,7 Гбит/с (API) + 27 Гбит/с CDN	6 Гбит/с (API)	4 узла L7 Gateway (10 GbE) + внешний CDN
Оперативная память кэша (RAM)	120 ГБ чистых данных	156 ГБ с фрагментацией	312 ГБ с репликацией ×2	12 узлов Redis (6 Master + 6 Replica × 32 ГБ)
Всего серверов кластера	64 сервера (штатно)	64 сервера (запас по CPU)	82 сервера (Auto-scaling)	Stateless: 12→30, БД: 18, Redis: 12, ES: 10, Инфра: 12

Этап 4. Архитектура верхнего уровня

> **Рассуждение кандидата вслух:**
> *«При проектировании верхнеуровневой архитектуры мы разделяем систему по бизнес-доменам и строго применяем принцип Database per Service: у каждого микросервиса — своё изолированное хранилище, подобранное под его профиль нагрузки. Никаких общих таблиц между сервисами! Там, где клиенту нужен мгновенный ответ (поиск, добавление в корзину, первичный резерв стока), мы используем синхронные вызовы (REST/gRPC). А все длинные цепочки — передачу заказа на склад, рассылку чеков, обновление поискового индекса и аналитику — развязываем асинхронно через шину событий Kafka».*

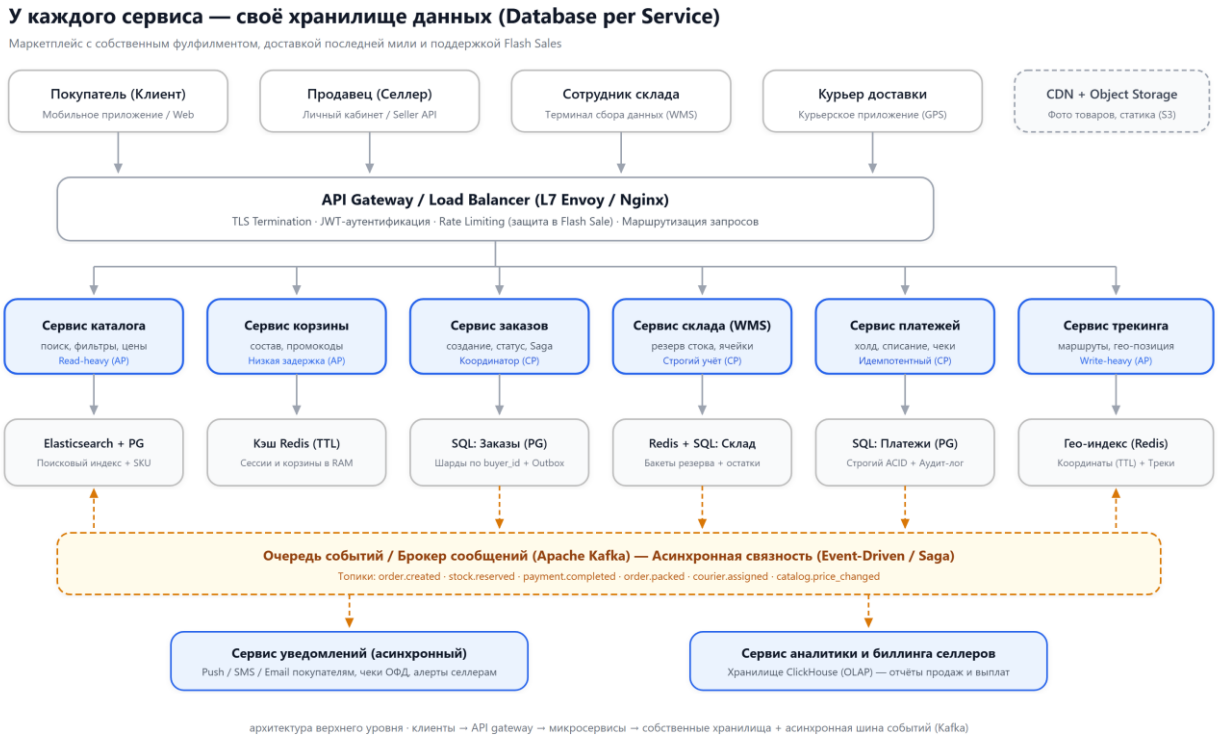


Рис. 1. Архитектура верхнего уровня маркетплейса: клиенты → API Gateway → сервисы → собственные хранилища (Database per Service) + шина событий Kafka

Назначение основных компонентов и обоснование выбора хранилищ

- **1. Клиентский уровень и CDN + Object Storage (S3):** четыре специализированных клиента (мобильное/веб-приложение покупателя, личный кабинет продавца, ТСД-приложение комплектовщика склада и мобильное приложение курьера). Все тяжёлые медиафайлы (фотографии товаров, видеообзоры, CSS/JS-бандлы) раздаются напрямую из гео-распределённой сети CDN поверх объектного хранилища S3, не нагружая API Gateway и сервисы.
- **2. API Gateway / Load Balancer (L7 Envoy / Nginx):** единая точка входа для всех внешних клиентов. Отвечает за TLS-терминацию, проверку JWT-токенов авторизации, маршрутизацию по микросервисам и, что особенно важно для маркетплейса, Rate Limiting и защиту от ботов при старте Flash Sales.
- **3. Сервис каталога и поиска (Catalog & Search Service):** обслуживает 90%+ всех запросов чтения. В качестве мастер-хранилища карточек SKU и цен использует PostgreSQL, для полнотекстового поиска с фасетными фильтрами (категория, бренд, цена, доставка) — кластер Elasticsearch, а для мгновенной отдачи популярных карточек — кэш Redis.
- **4. Сервис корзины (Cart Service):** хранит временный состав корзины покупателя и применённые промокоды. Поскольку корзина часто меняется и требует минимальной задержки, её единственное хранилище — Redis Cluster с настроенным TTL (например, 30 дней для авторизованных и 7 дней для гостей) и периодической персистенцией (AOF/RDB).
- **5. Сервис заказов (Order Service):** ключевой координатор оформления заказа (реализует паттерн Saga). Хранит заказы и их позиции в шардированном по buyer_id кластере PostgreSQL. Использует паттерн Transactional Outbox для гарантированной отправки событий о создании и смене статуса заказа в Kafka.
- **6. Сервис склада и резерва (Inventory & WMS Service):** отвечает за строгий учёт остатков товаров по региональным складам и адресным ячейкам, а также за маршруты комплектации. Использует двухуровневое хранилище: атомарные счётчики в Redis для мгновенного резерва стока (особенно в момент Flash Sale) и реляционную БД PostgreSQL как систему записи (System of Record) для адресного хранения.
- **7. Сервис платежей (Payment Service):** интегрируется с внешними платёжными шлюзами и банками-эквайерами (холд средств, списание, возвраты, фискализация чеков по 54-ФЗ). Использует выделенный PostgreSQL со строгим уровнем изоляции транзакций и проверкой уникального ключа идемпотентности (Idempotency Key).
- **8. Сервис доставки и трекинга (Delivery & Tracking Service):** назначает курьеров на собранные заказы, строит маршрутные листы и принимает поток GPS-координат (3 000 пингов/сек). Актуальная гео-позиция

каждого курьера хранится в Гео-индексе Redis (с TTL = 60 сек для мгновенного чтения клиентом), а история маршрутов асинхронно сбрасывается в колоночное/TimeSeries хранилище.

- **9. Брокер сообщений (Apache Kafka) и асинхронные сервисы:** обеспечивает слабую связность (loose coupling) микросервисов. Через топики Kafka работают Сервис уведомлений (Push/SMS/Email) и Сервис аналитики продавцов на базе OLAP-СУБД ClickHouse.

Этап 5. Глубокий разбор одного сложного узла

> **Рассуждение кандидата вслух:**
> *«Самый критичный и технически сложный узел нашей системы — это резервирование складских остатков и защита от оверселлинга (Overselling) в момент старта горячей распродажи (Flash Sale). Представьте: в 12:00:00 стартует распродажа 1 000 смартфонов со скидкой 50%, и в ту же секунду 50 000 человек одновременно нажимают кнопку „Оформить заказ". Давайте разберём, почему классический реляционный подход здесь гарантированно упадёт, и какая комбинация структур данных решает задачу за доли миллисекунды».*

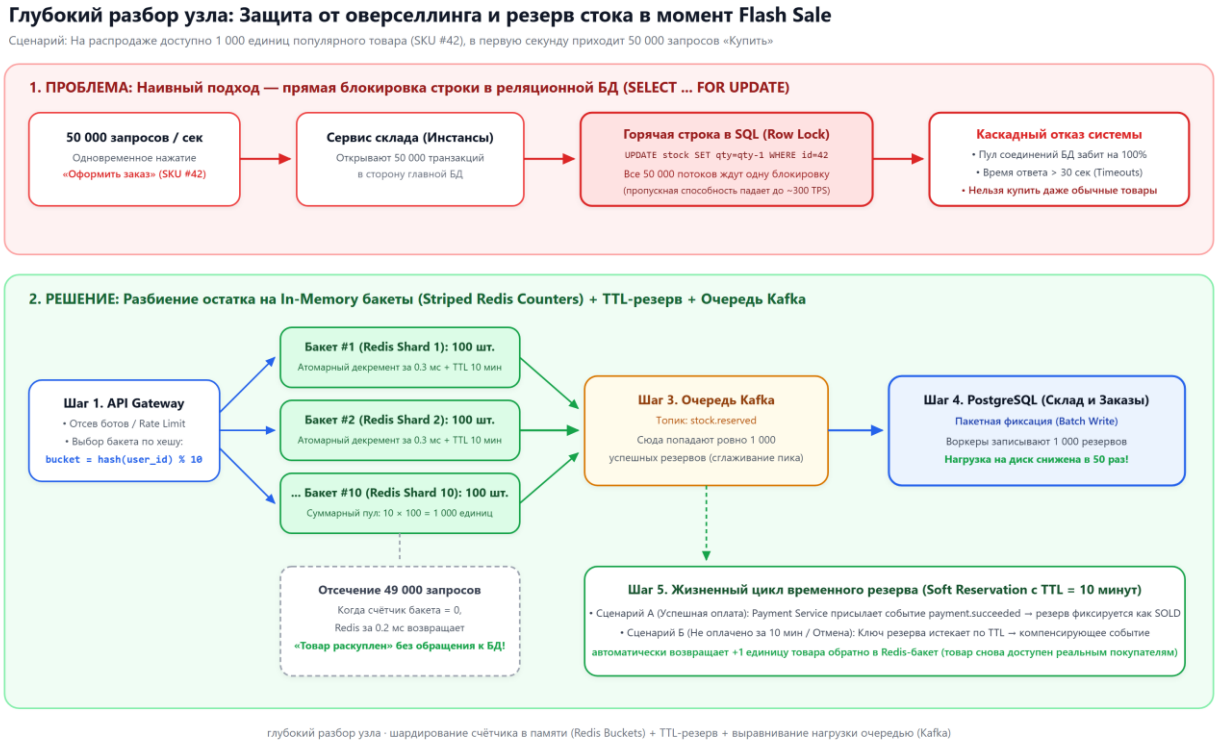


Рис. 2. Глубокий разбор узла: почему падает прямая блокировка строки в SQL и как работает схема с шардированными бакетами Redis, TTL-резервом и очередью Kafka

5.1. Проблема: почему наивный подход не работает при росте нагрузки

В наивной реализации разработчик хранит остаток товара в таблице реляционной БД PostgreSQL (warehouse_stock) и при нажатии покупателем кнопки «Оформить заказ» открывает транзакцию с пессимистичной блокировкой строки: SELECT available_qty FROM warehouse_stock WHERE product_id = 42 FOR UPDATE, проверяет, что остаток > 0, и выполняет UPDATE warehouse_stock SET available_qty = available_qty - 1. При нагрузке в 10–20 заказов в секунду на разные товары это прекрасно работает. Но в момент старта Flash Sale (50 000 запросов в секунду на один и тот же SKU #42) возникают сразу три фатальные проблемы:

- **1. Сериализация на «горячей строке» (Hot Row Lock Contention):** все 50 000 параллельных транзакций пытаются изменить одну и ту же физическую строку на диске. Пока первая транзакция не сделает COMMIT (включая синхронный сброс WAL-журнала на диск и сетевой round-trip, что занимает ~2–5 мс), остальные 49 999 потоков стоят в очереди ожидания блокировки. В итоге пропускная способность падает до 200–400 TPS, а хвост очереди ждёт ответа больше 100 секунд.

- **2. Исчерпание пула соединений и каскадное падение всей БД:** зависшие в ожидании блокировки запросы моментально выбирают весь пул соединений (Connection Pool) базы данных склада и заказов. В результате база перестаёт отвечать даже тем покупателям, которые в этот момент просто покупают обычные носки или книги, — ложится весь маркетплейс.
- **3. Дилемма «когда списывать сток — при создании заказа или при оплате»:** если списывать остаток только после успешной оплаты на стороне банка (через 1–2 минуты), то все 50 000 человек увидят, что товар есть, перейдут в банк и оплатят заказ. В итоге мы продадим 1 000 телефонов 50 000 раз (массовый оверселлинг) и разоримся на комиссиях за 49 000 возвратов. А если списывать остаток сразу при нажатии «Оформить заказ» без ограничения времени жизни резерва — конкуренты или боты за 1 секунду нажмут «Купить» 1 000 раз, не оплатят ни одного заказа, и реальные покупатели не смогут ничего купить.

5.2. Решение: шардированные In-Memory бакеты (Redis) + TTL-резерв + очередь Kafka

Чтобы решить задачу без блокировок диска и без риска оверселлинга, мы выносим «горячий» контур проверки и первичного резерва из реляционной БД в оперативную память и применяем четыре архитектурных приёма:

- **Приём 1. Предзагрузка остатка в память и атомарное списание в Redis:** перед стартом распродажи (или автоматически для любого горячего товара) доступный остаток загружается в кластер Redis. Проверка наличия, проверка того, что этот пользователь ещё не резервировал акционный товар, уменьшение счётчика на 1 и создание ключа временного резерва выполняются как единая неделимая (атомарная) операция в оперативной памяти за ~0,2–0,3 мс.
- **Приём 2. Разбиение счётчика на независимые слоты — бакеты (Inventory Bucketing / Striped Counters):** если на один ультра-популярный товар приходит 50 000–100 000 RPS, то даже один узел Redis (работающий в одном главном потоке) может стать узким местом. Поэтому мы применяем тот же приём декомпозиции, что и разбиение карты на гео-зоны в такси: делим общий остаток 1 000 штук на K = 10 независимых бакетов по 100 штук каждый (sku:42:bucket_0 … sku:42:bucket_9), которые лежат на 10 разных физических шардах Redis. Когда приходит запрос от покупателя, API Gateway вычисляет номер бакета по хешу его идентификатора: bucket_id = hash(user_id) % 10 и направляет запрос в конкретный шард.
- **Приём 3. Мягкий резерв с таймером жизни (Soft Reservation с TTL = 10 минут):** при успешном декременте бакета создаётся запись резерва reserve:{order_id} со временем жизни TTL = 10 минут. Покупателю гарантируется, что эта конкретная единица товара ждёт его оплаты 10 минут. Если за 10 минут событие успешной оплаты от Сервиса платежей не поступило, по истечении TTL срабатывает компенсирующий возврат: единица товара автоматически прибавляется (+1) обратно в соответствующий бакет Redis и снова становится доступной на витрине.
- **Приём 4. Выравнивание нагрузки на реляционную БД через очередь (Kafka Write-Behind):** реляционная БД PostgreSQL вообще не участвует в синхронном отсеве 50 000 запросов! Те 49 000 человек, которым не хватило товара, получают мгновенный ответ «Товар раскуплен» прямо из памяти Redis за 0,2 мс. А ровно те 1 000 запросов, которые успешно уменьшили счётчик в бакетах от 100 до 0, публикуют событие stock.reserved в топик Kafka. Фоновые воркеры Сервиса склада и Сервиса заказов вычитывают эти 1 000 событий из очереди в комфортном темпе (например, пачками по 100 штук за 1–2 секунды) и спокойно записывают их в PostgreSQL без единого конфликта блокировок.

5.3. Почему это работает: какой выигрыш получает система

- **1. Выигрыш в скорости на два порядка (0,3 мс вместо 50+ мс):** операция декремента в оперативной памяти Redis не требует обращения к диску и ожидания fsync транзакционного лога, поэтому выполняется за доли миллисекунды.
- **2. Горизонтальный параллелизм и отсутствие конфликтов (Zero Lock Contention):** благодаря разбиению счётчика на 10 шардированных бакетов нагрузка распределяется равномерно по 10 ядрам разных серверов Redis (по 5 000 RPS на шард при запасе прочности шарда до 80 000 RPS). Если в каком-то одном бакете на самом излёте распродажи товар закончился на долю секунды раньше, чем в соседнем, клиент получает быстрый retry в соседний непустой бакет.
- **3. Снижение нагрузки на дисковую БД в 50 раз (Load Leveling):** вместо 50 000 конкурирующих транзакций в секунду до PostgreSQL доходят ровно 1 000 валидных резервов, выровненных во времени очередью Kafka. Остальной маркетплейс продолжает работать так, будто никакой пиковой атаки не было.

Этап 6. Проектирование базы данных и ER-диаграмма

> **Рассуждение кандидата вслух:**
> *«Построим ER-диаграмму данных нашей системы. Как и в классическом маркетплейсе доставки, центральной стержневой сущностью здесь выступает Заказ (Order): именно он в момент покупки связывает воедино Покупателя, конкретный Склад фулфилмента, Курьера доставки, Платёж и купленные Товары продавцов. Особое внимание уделим связям многие-ко-многим (N:M): один заказ содержит много товаров, а один товар входит во множество заказов — поэтому между ними обязательно появляется таблица Позиция заказа (OrderItem). Аналогично связь между Товаром и Складом развязывается через таблицу Складской остаток (WarehouseStock)».*

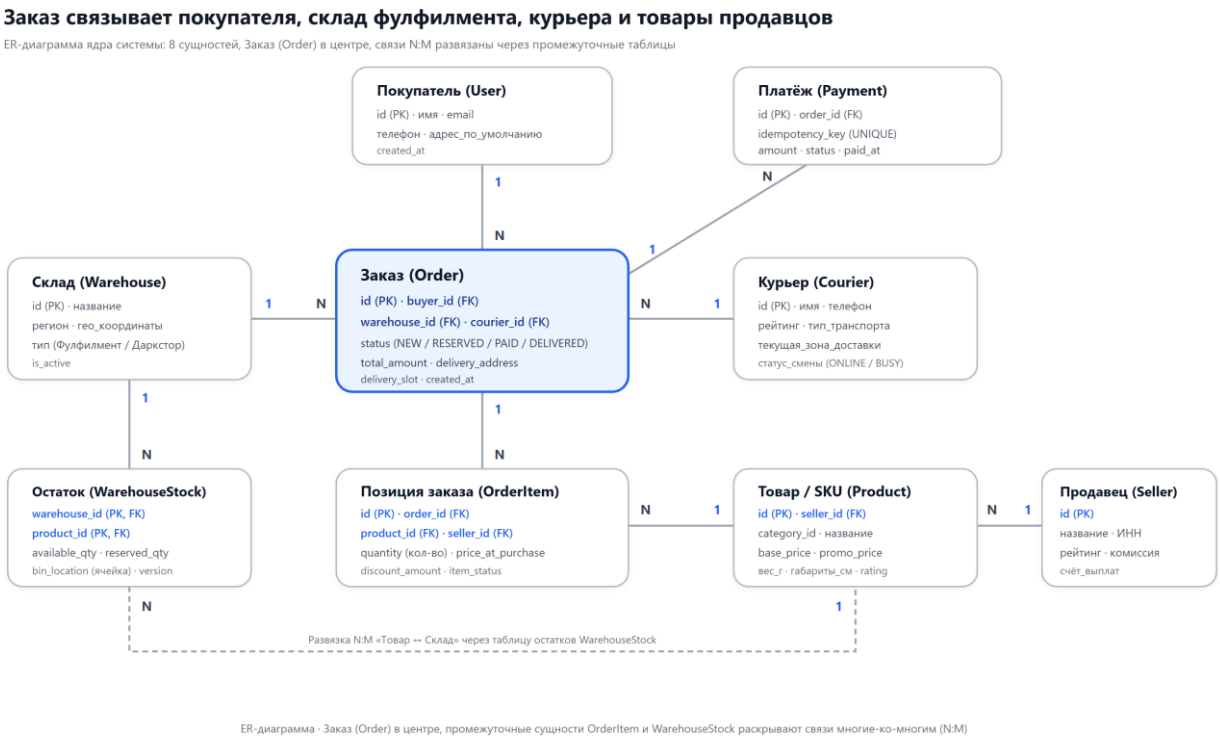


Рис. 3. ER-диаграмма базы данных маркетплейса: центральная сущность Заказ (Order) и развязка связей N:M через OrderItem и WarehouseStock

Проверка кардинальности всех связей (1:N и развязка N:M)

- **1. Покупатель (User) ↔ Заказ (Order) — связь 1 : N:** один покупатель может оформить множество заказов за всё время пользования сервисом, но каждый конкретный заказ принадлежит ровно одному покупателю (внешний ключ buyer_id в таблице Order).
- **2. Склад фулфилмента (Warehouse) ↔ Заказ (Order) — связь 1 : N:** с одного регионального склада отгружаются миллионы заказов, при этом конкретный заказ (отправление) комплектуется и отгружается с одного выбранного склада (внешний ключ warehouse_id в таблице Order; если покупатель заказал товары с двух разных складов, система при чекауте создаёт два дочерних заказа-отправления).
- **3. Курьер (Courier) ↔ Заказ (Order) — связь 1 : N:** один курьер за смену развозит множество заказов, а конкретный заказ на этапе последней мили закреплён за одним курьером (внешний ключ courier_id в таблице Order).
- **4. Заказ (Order) ↔ Платёж (Payment) — связь 1 : N:** по одному заказу может быть предпринято несколько попыток оплаты (например, первая отклонена банком из-за нехватки средств, вторая успешна, третья — частичный возврат), поэтому таблица Payment ссылается на заказ через order_id (FK).
- **5. Заказ (Order) ↔ Товар (Product) — логическая связь N : M, развязанная через Позицию заказа (OrderItem):** в одном заказе может быть несколько разных товаров, и один и тот же популярный товар покупается в тысячах разных заказов. Прямая связь N:M в реляционной БД невозможна, поэтому вводится промежуточная сущность OrderItem (связи: Order 1 → N OrderItem и Product 1 → N OrderItem). Критически

важная деталь: в OrderItem обязательно сохраняется атрибут price_at_purchase (цена на момент покупки), чтобы последующее изменение цены продавцом в таблице Product не исказило историю уже оформленных заказов!

- **6. Склад (Warehouse) ↔ Товар (Product) — логическая связь N : M, развязанная через Остаток (WarehouseStock):** на одном складе хранятся сотни тысяч разных товаров, а один и тот же товар (SKU) может одновременно лежать на 15 региональных складах по всей стране. Промежуточная таблица WarehouseStock имеет составной первичный ключ (warehouse_id, product_id) и хранит доступное количество (available_qty), зарезервированное количество (reserved_qty) и код адресной ячейки (bin_location).
- **7. Продавец (Seller) ↔ Товар (Product) — связь 1 : N:** один продавец размещает на площадке множество карточек товаров (SKU), а каждая конкретная карточка SKU принадлежит одному продавцу (внешний ключ seller_id в таблице Product).

Сводная спецификация таблиц, ключей и индексов

Сущность (Таблица)	Первичный ключ (PK)	Внешние ключи (FK)	Ключевые атрибуты и типы	Индексы и ключ шардирования
Order (Заказ — центр)	id (UUID / BIGINT)	buyer_id → User.id warehouse_id → Warehouse.id courier_id → Courier.id	status (ENUM), total_amount (NUMERIC(12,2)), delivery_address (TEXT), delivery_slot (TSTZRANGE), created_at	Шардирование по buyer_id; составной индекс (buyer_id, created_at DESC); индекс (warehouse_id, status) для сборки.
OrderItem (Позиция заказа)	id (BIGINT)	order_id → Order.id product_id → Product.id seller_id → Seller.id	quantity (INT), price_at_purchase (NUMERIC(12,2)), discount_amount (NUMERIC), item_status (ENUM)	Ко-локация на одном шарде с Order; индекс (order_id); аналитический индекс в DWH по (seller_id, product_id).
Payment (Платёж)	id (UUID)	order_id → Order.id	idempotency_key (VARCHAR UNIQUE), amount (NUMERIC(12,2)), provider (VARCHAR), status (ENUM), paid_at	UNIQUE индекс по (idempotency_key) для защиты от двойного списания; индекс по (order_id).
User (Покупатель)	id (BIGINT)	—	name (VARCHAR), phone (VARCHAR UNIQUE), email (VARCHAR), default_address (JSONB), created_at	UNIQUE индекс по phone; хэш-шардирование по id.
Product (Товар / SKU)	id (BIGINT)	seller_id → Seller.id	category_id (INT), title (VARCHAR), base_price (NUMERIC), promo_price (NUMERIC), weight_g (INT), rating (NUMERIC(3,2))	Индекс (seller_id); индекс (category_id); полнотекстовый и фасетный индекс синхронизируется в Elasticsearch.
WarehouseStock (Остаток)	(warehouse_id, product_id) — составной PK	warehouse_id → Warehouse.id product_id → Product.id	available_qty (INT CHECK ≥ 0), reserved_qty (INT CHECK ≥ 0), bin_location (VARCHAR), version (BIGINT)	PK-индекс (warehouse_id, product_id); индекс (product_id) для поиска складов с наличием товара.
Warehouse (Склад)	id (INT)	—	name (VARCHAR), region_code (VARCHAR), geo_point (POINT), warehouse_type (ENUM), is_active (BOOL)	GiST гео-индекс по geo_point для выбора ближайшего склада к покупателю.
Seller (Продавец) и Courier (Курьер)	id (BIGINT)	—	Seller: name, inn, commission_pct, payout_account. Courier: name, phone,	Seller: UNIQUE (inn). Courier: индекс (shift_status, current_zone_id) + Гео-

			vehicle_type, rating, shift_status.	индекс в Redis.

Этап 7. Масштабируемость, надёжность и компромиссы

> **Рассуждение кандидата вслух:**
> *«В распределённой системе при сетевом разделении (Partition Tolerance) мы не можем одновременно гарантировать абсолютную консистентность и 100% доступность для всех узлов сразу. Поэтому грамотный архитектор делит систему на два режима: там, где речь идёт о физических деньгах и штучном товаре на полке, мы выбираем CP (лучше вернуть ошибку или подождать, чем продать несуществующий товар или списать деньги дважды). А там, где пользователь просто листает витрину, читает отзывы или смотрит на машинку курьера на карте, мы выбираем AP (лучше показать цену или координату двухсекундной давности, чем показать белый экран ошибки)».*

7.1. Выбор между CP и AP (CAP-теорема) по подсистемам маркетплейса

Подсистема / Сервис	Режим по CAP	Обоснование выбора и бизнес-компромисс
1. Сервис склада и резервирования остатков (Inventory Service)	CP (Consistency + Partition Tolerance)	Физическое количество товара на полке конечно. Если из-за сетевого рассинхрона два узла продадут одну и ту же последнюю единицу товара разным покупателям (Overselling), бизнес понесёт прямые финансовые и репутационные потери. При недоступности мастера шарда остатков лучше временно заблокировать покупку этого конкретного SKU, чем допустить отрицательный баланс.
2. Сервис платежей и биллинга (Payment Service)	CP (Consistency + Partition Tolerance)	Финансовые транзакции требуют строгих гарантий ACID, линейной сериализуемости и защиты от двойного списания. Неконсистентное состояние платежа недопустимо по закону и правилам банковского эквайринга.
3. Сервис заказов (Order Service — ядро оформления)	CP (на запись заказа) + Eventual Consistency (между сервисами через Saga)	Сам факт создания заказа и смены его статуса внутри БД заказов должен быть строго консистентным (CP). При этом связка между Заказом, Складом и Оплатой строится через паттерн Saga с компенсирующими транзакциями (если оплата не прошла — заказ отменяется, а резерв возвращается на склад).
4. Сервис каталога, поиска и отзывов (Catalog & Search Service)	AP (Availability + Partition Tolerance)	Витрина — главное лицо маркетплейса. Покупатель должен всегда иметь возможность искать товары и открывать карточки. Если продавец изменил описание товара или появился новый отзыв, задержка его появления в поисковом индексе Elasticsearch на 1–3 секунды совершенно незаметна для бизнеса.
5. Сервис корзины (Cart Service — до момента чекаута)	AP (Availability + Partition Tolerance)	Покупатель должен мочь добавить товар в корзину в любой момент без задержек. Строгая проверка наличия товара и актуальной цены всё равно выполняется в момент перехода к оформлению заказа (Checkout).
6. Сервис гео-трекинга курьера (Delivery Tracking Service)	AP (Availability + Partition Tolerance)	Если на карте покупателя иконка курьера покажет позицию 5-секундной давности из-за потери одного пакета или чтения с реплики, пользовательский опыт не пострадает. Главное — высокая доступность и скорость отдачи карты.

7.2. Узкие места системы (Bottlenecks) и архитектурные решения

Узкое место (Bottleneck)	Почему возникает проблема	Архитектурное решение и механизм защиты
1. Экстремальный всплеск нагрузки при	В одну секунду по пуш-уведомлению приходят сотни	• Вынос статики и пререндеренных страниц акций на CDN. • Rate Limiting и защита от ботов на API Gateway.

старте Flash Sale (11.11 / Чёрная пятница)	тысяч пользователей: трафик чтения вырастает в 10 раз, а запросы на покупку акционных SKU — в 100 раз.	• Горизонтальный автоскейлинг (HPA) stateless-сервисов. • Двухуровневый резерв стока: шардированные бакеты в памяти Redis отсекают 98% лишних запросов, а очередь Kafka сглаживает запись в PostgreSQL (Load Leveling).
2. Проблема «горячего товара» или «горячего продавца» (Hot Key / Celebrity SKU)	Даже при шардировании по product_id один вирусный товар блогера перегружает конкретный шард БД и кэша (перекос нагрузки — Shard Skew).	• Для чтения: локальный In-Memory кэш (L1 Caffeine/Go-cache с TTL = 2–3 сек) прямо внутри оперативной памяти инстансов Сервиса каталога перед походом в Redis (L2). • Для записи резервов: разбиение счётчика одного SKU на N = 10–20 виртуальных слотов-бакетов с суффиксами (sku_id#1..#10), распределённых по разным узлам Redis.
3. Рассинхрон данных между микросервисами Заказов, Склада и Платежей при сбое сети	При архитектуре Database per Service нельзя сделать единый SQL-транзакционный COMMIT сразу в три разные базы. Если БД заказа сохранила строку, но упала сеть до отправки сообщения в Kafka, система зависнет в полусобранном состоянии.	• Паттерн Transactional Outbox: сервис записывает сам заказ и событие для Kafka в одну локальную транзакцию PostgreSQL (в таблицу outbox), откуда процесс CDC (Debezium) гарантированно доставляет событие в Kafka. • Оркестрация распределённой транзакции через паттерн Saga с компенсирующими шагами (отмена резерва по TTL или при отказе оплаты).
4. Деградация или таймауты внешнего платёжного шлюза (банка-эквайера)	Внешний банк начинает отвечать по 15–20 секунд; потоки Сервиса платежей и Сервиса заказов блокируются в ожидании HTTP-ответа, вызывая каскадный отказ.	• Асинхронное взаимодействие с банком через вебхуки и очередь опроса. • Паттерн Circuit Breaker (Resilience4j) с жёстким таймаутом (3 сек) и автоматическим переключением трафика (Fallback Routing) на резервный банк-эквайер. • Изоляция пулов потоков (Bulkhead Pattern).
5. Лаг асинхронной репликации БД (Read-after-Write Inconsistency)	Покупатель только что оформил заказ (запись ушла в Master БД) и через 100 мс перешёл на экран «Мои заказы», который читает с асинхронной Read-реплики, куда запись ещё не доехала — покупатель не видит свой заказ и пугается.	• Паттерн Read-Your-Own-Writes (Pinning to Master): после любой операции записи пользователя в течение 5 секунд все его запросы чтения (по флагу в сессии/куке) принудительно маршрутизируются на Master-узел БД либо читают свежий объект напрямую из кэша Redis.
6. Дублирование запросов при плохом мобильном интернете (Повторная оплата / дубли заказа)	Клиент в метро нажимает «Оплатить», запрос доходит до сервера, но ответ теряется; приложение делает retry, и создаётся второй такой же заказ или списываются вторые деньги.	• Сквозной ключ идемпотентности (Idempotency Key — UUID, генерируемый клиентом при открытии экрана чекаута). • Сервисы Заказов и Платежей проверяют ключ в Redis/PostgreSQL (UNIQUE индекс): при повторном запросе с тем же ключом новая операция не выполняется, а возвращается сохранённый результат первой.

Итоговое резюме кандидата в конце системного дизайн-интервью:
«Подведём итог. Мы спроектировали масштабируемую архитектуру маркетплейса с фулфилментом, способную обслуживать 10 млн активных пользователей в сутки (до 14 000 RPS в вечерний пик и до 60 000 RPS в момент старта Flash Sale) на кластере из ~64–82 серверов. Мы разделили контуры чтения (AP: CDN, Redis, Elasticsearch) и контуры денег и товарных остатков (CP: шардированный PostgreSQL, атомарные бакеты резерва в Redis и паттерн Saga + Outbox поверх Kafka), полностью защитив бизнес как от оверселлинга на распродажах, так и от каскадных падений базы данных».